



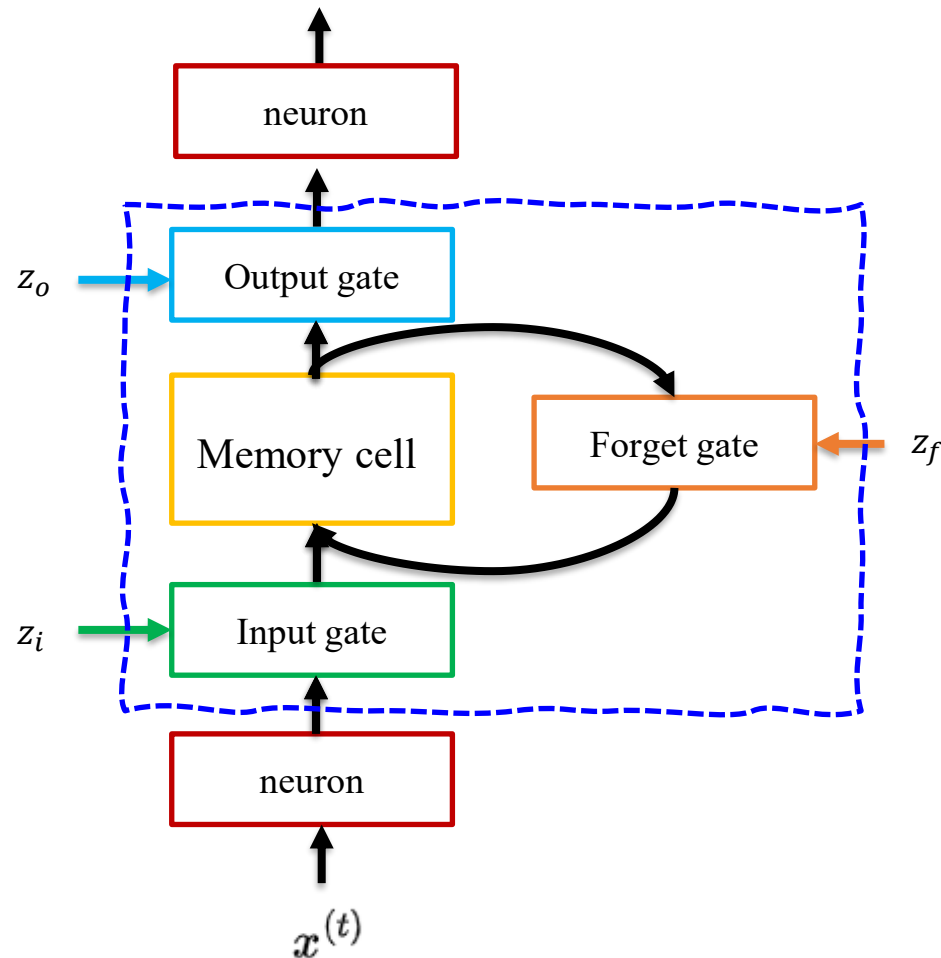
國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

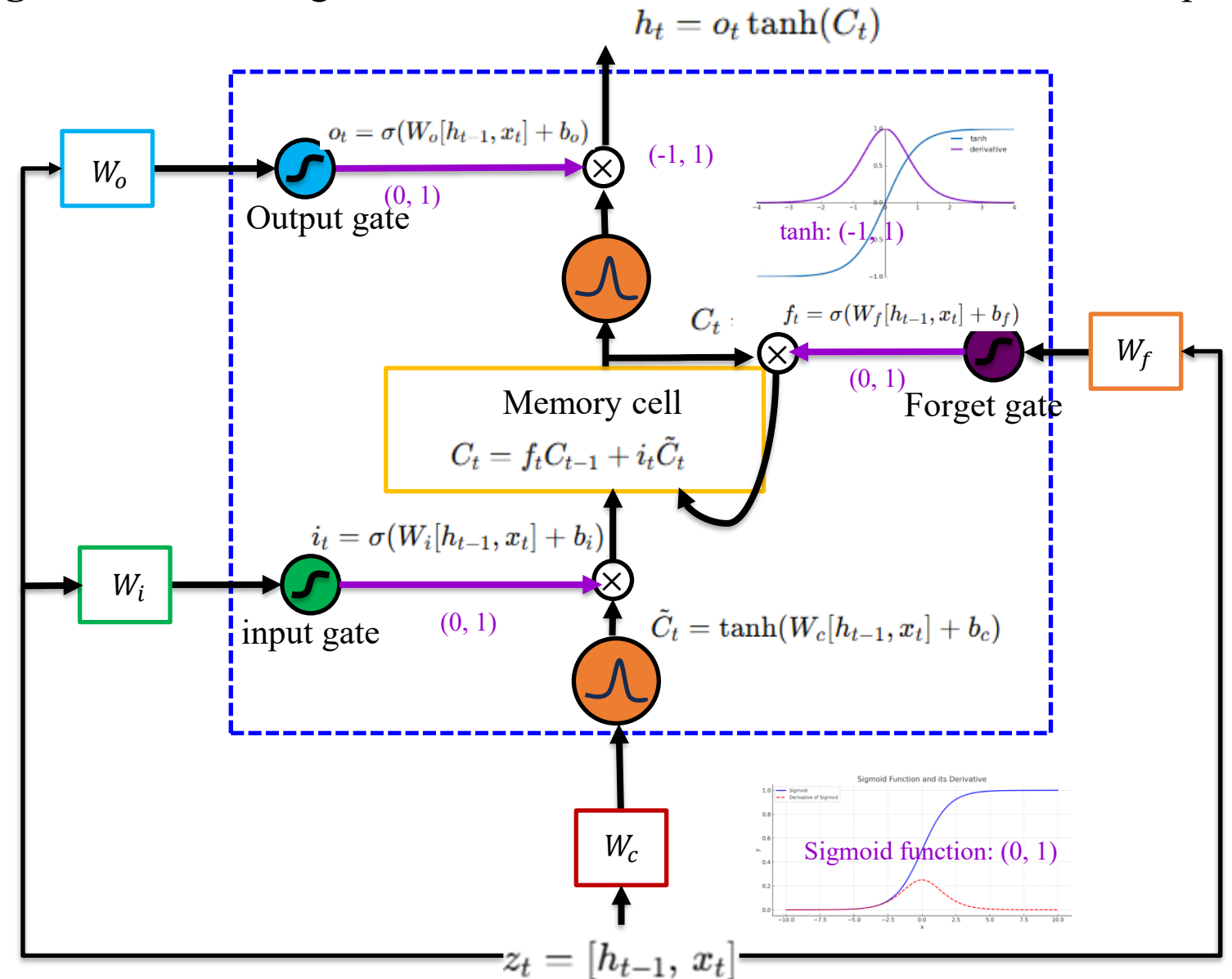
Ching-I Huang

Sequence Model

- A vanilla RNN “forgets” long-term dependencies because each update **overwrites the hidden state**.
- This repeated overwriting causes **vanishing gradients**, making the model lose information from distant time steps.
- Long Short-Term Memory (LSTM)



$z_t = [h_{t-1}, x_t]$ concatenates previous hidden state and current input.





Sequence Model

- Long Short-Term Memory (LSTM)

1. Generate new candidate content

$$z_t = [h_{t-1}, x_t]$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

2. Control how much new information is written

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

3. Retain relevant past memory

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

4. Accumulate and preserve memory over time

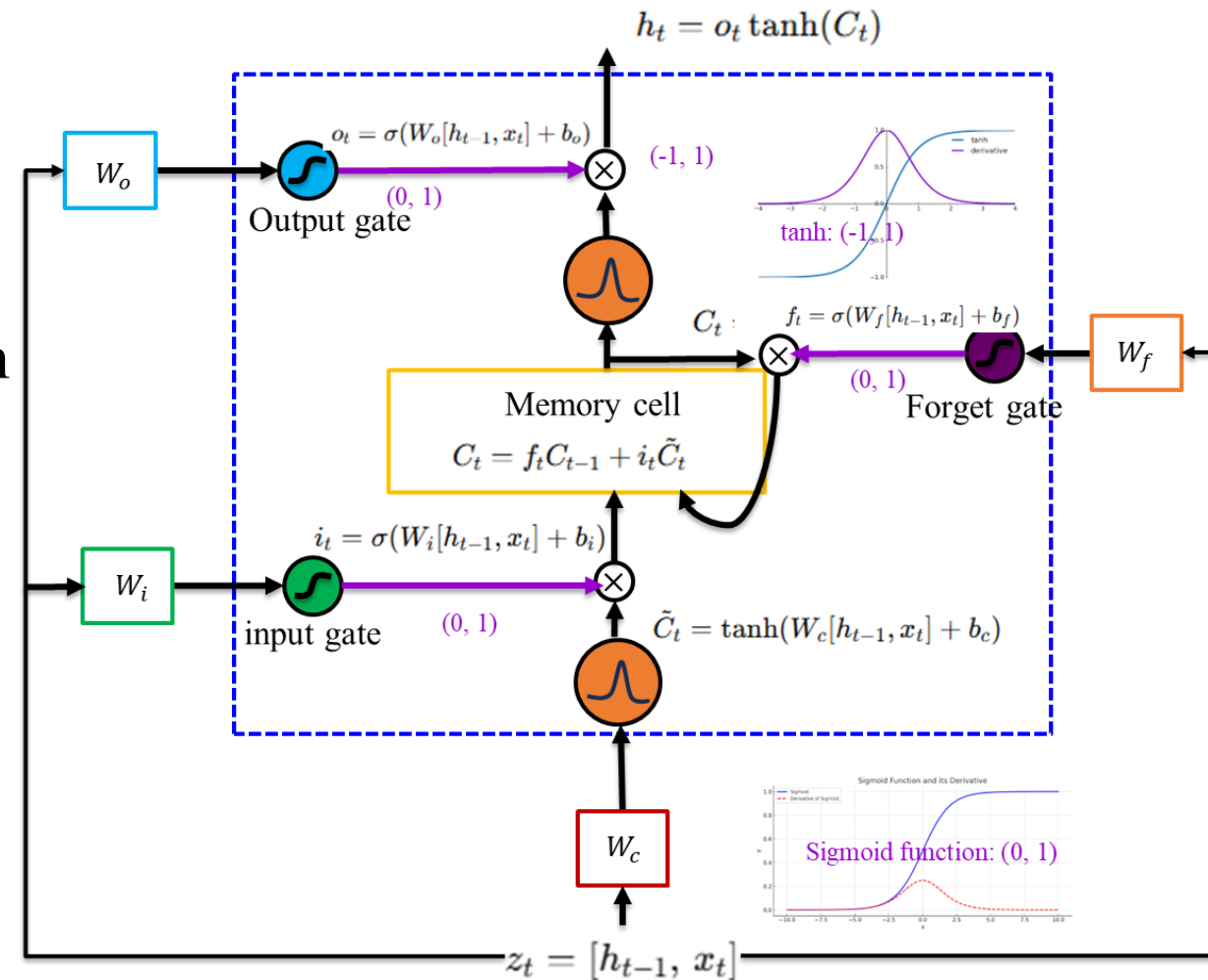
$$C_t = f_t C_{t-1} + i_t \tilde{C}_t \quad \text{Element-wise product}$$

5. Control the exposure of cell information

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

6. Produce the compressed hidden state for output

$$h_t = o_t \tanh(C_t)$$



Sequence Model

- Long Short-Term Memory (LSTM)

1. Long-Term Information Storage

- If certain information remains useful, LSTM keeps $f_t \approx 1, i_t \approx 0$:

$$\begin{aligned} C_t &= f_t C_{t-1} + i_t \tilde{C}_t \\ &= 1 \times C_{t-1} + 0 \times \tilde{C}_t = C_{t-1} \end{aligned}$$

- This allows the memory to persist unchanged across many time steps without gradient decay.
- Gating mechanism ensures that **important contextual information** (e.g., subject, topic, or long-term pattern).

2. Forget Gate Bias Initialization Trick

- During training, the **forget gate** f_t controls how much past information is retained.
- Set the forget gate bias b_f to a **positive value** (e.g., 1 or 2).

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

This pushes the initial toward $\sigma(b_f) \approx 0.7\text{--}0.9$, meaning the model **initially remembers more** and forgets slowly.

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \\ C_t &= f_t C_{t-1} + i_t \tilde{C}_t \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ h_t &= o_t \tanh(C_t) \end{aligned}$$



Sequence Model

- Long Short-Term Memory (LSTM)

3. Different Activations

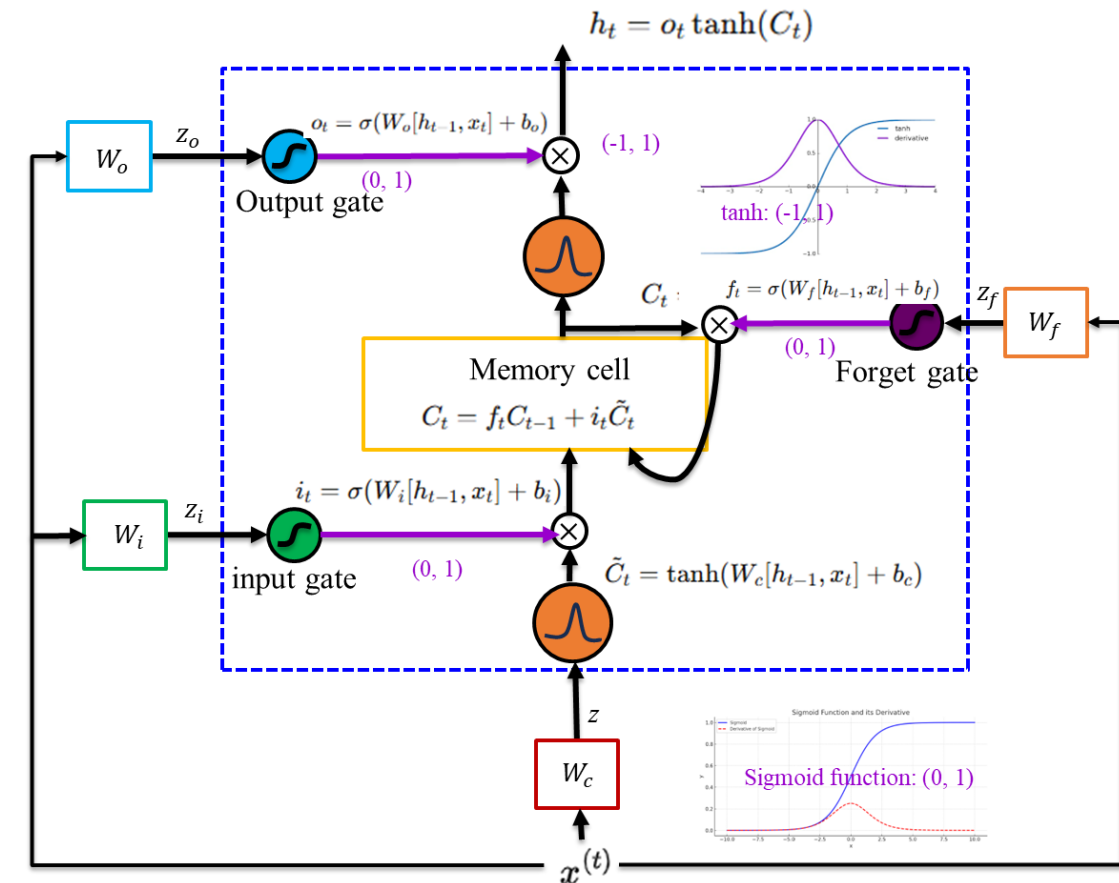
- Controlled information flow (via sigmoid)

Sigmoid gates act as soft binary switches, deciding how much to keep, write, or output.

- Rich representational capacity (via tanh)

At candidate memory, tanh allows both positive and negative values, representing directional information(e.g., increase vs. decrease in a feature).

- Compresses memory state before output



Sequence Model

- Long Short-Term Memory (LSTM)

4. Gradient Flow Through the Cell State

$$\begin{cases} \frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \cdot \frac{\partial c_t}{\partial c_{t-1}} \\ C_t = f_t C_{t-1} + i_t \tilde{C}_t \end{cases}$$
$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \cdot f_t \quad \longleftrightarrow \quad \frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial h_t} \cdot W_{hh} \cdot \sigma'(a_t) \quad \text{RNN}$$

When $f_t \approx 1$, then:

$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \cdot 1 = \frac{\partial L}{\partial c_t}$$

Constant Error Carousel (CEC):

The gradient **passes directly** without shrinking.

The gradient can **remain constant across many time steps**.

5. Cannot Solve Gradient Explosion

When $f_t \approx 1$, $\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ can still amplify signals.

If weight matrices W or U are too large, gradients may become unstable.

Even if the cell state gradient doesn't vanish, it can suddenly **explode** at certain time steps.



Generative models

Instead of having separate unsupervised and supervised components in the model, one can construct models in which a generative model of either $P(x)$ or $P(x, z)$ shares parameters with a discriminative model of $P(z|x)$. This allows for a trade-off between the supervised criterion $-\log P(z|x)$ and the unsupervised or generative criterion (such as $-\log P(x)$ or $-\log P(x, z)$). The generative criterion then represents a particular form of prior belief about the solution to the supervised learning problem (see Sec. 7.6 in [2]).

The model transforms samples of latent variables z into samples x or into distributions over samples x using a differentiable function $g(z; \theta^{(g)})$, which is typically represented by a neural network (Sec. 20.10.2 in [2]).

To generate a sample from the model, the VAE first draws a sample z from the code distribution $p(z)$. This sample is then passed through a differentiable generator network $g(z)$. Finally, x is sampled from a distribution $p(x; g(z)) = p_\theta(x|z)$. During training, however, an approximate inference network (or encoder) $q(z|x)$ is used to obtain z ; $p_\theta(x|z)$ is then interpreted as a decoder network (see Sec. 20.10.3 in [2]).

The objective is to approximate the posterior distribution $p_\phi(z|x)$ by using a simpler distribution $q_\phi(z|x)$, where z represents the latent variables and x represents the observed data. The variational inference approach is used to minimize the divergence between the true posterior $p_\phi(z|x)$ and the approximate posterior $q_\phi(z|x)$, typically using the Kullback-Leibler (KL) divergence.

